Genetic Algorithm (GA)

Lecture 1: Chromosome Representation and Fitness

1. Genetic Algorithm

- A genetic algorithm is a **search algorithm** based on **optimization** techniques and **heuristic** functions. It follows the basic ideas of genetics and **natural selection**: in every generation, the best individuals are chosen and the rest are dropped.
- A genetic algorithm is a **stochastic search algorithm**, which means it uses randomness while searching.
- Genetic algorithms find the **optimal** solution, or a solution close to the optimal one (near optimal). They are used for **NP-hard** problems.

2. Genetic Algorithm Fundamentals

Term	Meaning
Population	The set of individuals (solutions) of the problem.
Chromosome	One individual, which means one solution of the problem.
Gene	One element of a chromosome. Every chromosome is made of a set of genes, so each cell of the chromosome is called a gene.
Allele	The value that a gene takes in a specific chromosome.
Fitness function	A function that measures how good a chromosome (solution) is. It is different from one problem to another.
Genetic operators	Crossover (mating), selection and mutation.

3. GA Iteration

Term	Meaning
Phenotype	The chromosome as it looks in the real world (the real solution).
Genotype	The chromosome written as genes for the given problem.

The steps of one GA iteration:

1. Population initialization
2. Fitness function calculation (*)
3. Selection for crossover
4. Crossover
5. Mutation
6. Survivor selection, then go back to step 2 (*)

This process is repeated until the stop condition is achieved.

4. Chromosome (Genotype) Representation

Binary Representation

1	0	0	1	0

Example, the 0/1 knapsack problem: `1` means the item is taken, and `0` means it is not taken.

Integer Representation

2	9	5	3	7

Example, the knapsack problem: we took 2 pieces of the first item. If the first item is keys, we took 2 keys. From the second item we took 9 pieces, and so on.

Real Representation

0.4	0.5	1.0	0.3	0.7

Example, the knapsack problem: we took 0.4 of the first item and 0.5 of the second item. From the third item we took 1.0, which means we took the whole item.

Permutation Representation

2	1	3	4	5

Example, the travelling salesman problem: each number represents a city, and its position shows the order in which the city is visited.

5. The Fitness Function: MAX or MIN?

A random chromosome is almost always a bad solution. Before the GA can improve anything, it must be able to **measure** how good each chromosome is. This is the job of the **fitness function** (step 2 of the GA iteration).

A fitness function takes one chromosome and gives back **one number**. Before you write it, ask yourself one question: **is a bigger number better, or a smaller number better?**

	Maximize (MAX)	**Minimize (MIN)**
The number is...	a score. More is better.	a cost or a count of mistakes. Less is better.
Real life	your mark in an exam	the number of mistakes in a dictation
Best value	often a known top value, like 11 out of 11	often 0 (no mistakes at all)
Python picks the best with	`max()`	`min()`

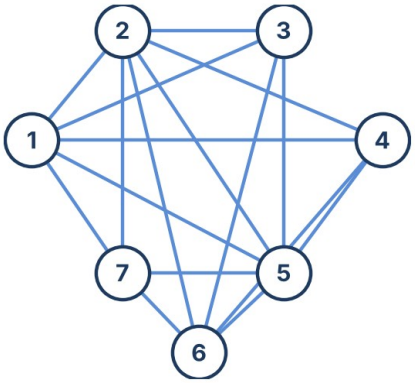
How do you know which one? **Read the goal of the problem carefully.** Words like “as few as possible”, “as close as possible”, “zero clashes” or “as little waiting as possible” tell you to **minimize**. Words like “as many as possible” or “the most correct” tell you to **maximize**.

6. The Problems

Read each problem carefully. Before you write any code, discuss with your group: **what should one gene mean? Is the fitness MAX or MIN?** Every problem has the same questions.

Problem 1: Graph Coloring

We have the following graph, and we want to color the nodes using the least number of colors, so that adjacent nodes (nodes joined by an edge) do not have the same color.




```
nodes = [1, 2, 3, 4, 5, 6, 7]

# pairs of nodes joined by an edge
edges = [(1, 2), (1, 3), (1, 4), (1, 5), (1, 7), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7),
         (3, 5), (3, 6), (4, 5), (4, 6), (5, 6), (5, 7), (6, 7)]

# at most one color per node
colors = ["Red", "Green", "Blue", "Yellow", "Purple", "Orange", "Pink"]
```

Goal: Color every node, using as few colors as possible, so that no edge joins two nodes of the same color.

Questions

1. Suggest a chromosome representation. What does one gene mean?
2. Write a fitness function. Is it a **MAX** or a **MIN** problem?
3. Write the Python code and test it on a small random population.

Problem 2: The Inheritance

A grandfather leaves 12 items to his two grandchildren, Lina and Omar. Nothing can be sold or cut in half. Each item goes to exactly one of them. The family wants the total values to be as close to each other as possible, so nobody feels cheated.

```
items = {
    "Car": 12000, "Piano": 7000, "Watch": 1500, "Painting": 4300,
    "Carpet": 2100, "Gold ring": 3200, "Books": 900, "Sewing machine": 450,
    "Dining table": 1800, "Bicycle": 350, "Camera": 1250, "Coin collection": 5600,
}
```

Goal: Minimize | total of Lina − total of Omar |

Questions

1. Suggest a chromosome representation. What does one gene mean?
2. If a third grandchild appeared, would your representation still work?
3. Write a fitness function. Is it a **MAX** or a **MIN** problem?
4. Write the Python code and test it on a small random population.

7. The Solutions

Every solution answers the same two questions:

- **Why is this chromosome representation a good one?**
- **Is the fitness MAX or MIN, and why did we choose this fitness?**

Every program has the same three parts, so you can compare them easily:

Function	What it does
`random_chromosome()`	Creates one random chromosome.
`decode(chromosome)`	Shows the chromosome as a real solution that a human can read.
`fitness(chromosome)`	Gives back one number: how good the chromosome is.

All the programs are in the folder `GA problems/code`. We use `random.seed()` so all students see the same output.

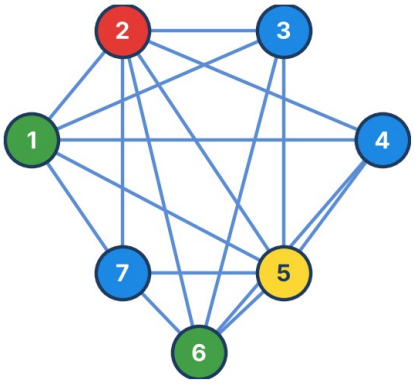
Solution 1: Graph Coloring

1. Chromosome representation: integer

Use **one gene per node**. The value of the gene is the **color number** of that node. The chromosome has **7 genes**.

Node	1	2	3	4	5	6	7
Gene	1	0	2	2	3	1	2
Color	Green	Red	Blue	Blue	Yellow	Green	Blue

Here is one example chromosome. It uses 4 colors, and no edge joins two nodes of the same color:



Why is this a good representation?

- Every chromosome gives **every node exactly one color**. There is no chromosome that means nothing.
- A gene can be any number from 0 to 6, because in the worst case each of the 7 nodes needs its own color. The GA then looks for chromosomes that use fewer colors.
- **Why not binary?** 7 colors need 3 bits per node ($2^3 = 8$ values). The value `111` would be an eighth color that does not exist, so some chromosomes would be invalid.

2. Fitness function

```
fitness = bad_edges × 10 + colors_used
```

MIN Lower is better. The fitness is a cost.

Why this fitness? The goal has two parts, and they are not equally important:

- No edge may join two nodes of the same color. This is a **must**.
- Use as few colors as possible. This is a **wish**.

So every bad edge costs 10 points, and every color costs only 1 point. We have only 7 colors, so one bad edge costs more than all the colors together. This means **any wrong coloring always loses against any correct coloring**. This trick is called a **penalty**. For example, a coloring with no bad edges and 4 colors has fitness 0 × 10 + 4 = 4.

3. Python code

1_graph_coloring.py

```
"""
Problem 1: Graph Coloring

Color the cities on a map with as few colors as possible,
so that two connected cities never have the same color.

Representation: integer, one gene per city (gene = color number 0 to 6)
Fitness:        MINIMIZE  bad_edges * 10 + colors_used

Run:  python 1_graph_coloring.py
"""

import random

# The map: each node is a city, and each edge connects two cities
nodes = [1, 2, 3, 4, 5, 6, 7]
edges = [(1, 2), (1, 3), (1, 4), (1, 5), (1, 7), (2, 3), (2, 4), (2, 5), (2, 6), (2, 7),
         (3, 5), (3, 6), (4, 5), (4, 6), (5, 6), (5, 7), (6, 7)]

# At most one color per city
colors = ["Red", "Green", "Blue", "Yellow", "Purple", "Orange", "Pink"]

# One gene per city, so the chromosome has 7 genes
N = len(nodes)

# The gene number of each city, for example city 1 -> gene 0
gene_of = {node: i for i, node in enumerate(nodes)}

# One bad edge costs more than all 7 colors together
PENALTY = 10


# ---------- The map ----------
def neighbors(node):
    # Find all cities that have an edge to this city
    result = []
    for a, b in edges:
        if a == node:
            result.append(b)
        elif b == node:
            result.append(a)
    return sorted(result)


def print_map():
    # Show how the cities are connected
    print("The map (how the cities are connected):")
    for node in nodes:
        print(f"  City {node} -> {neighbors(node)}")


def print_colors():
    # Show the colors and the gene value of each one
    print("The colors we can use (gene value = color):")
    for value, name in enumerate(colors):
        print(f"  {value} = {name}")
```

```
# ---------- Representation ----------
def random_chromosome():
    # Give every city a random color number from 0 to 6
    return [random.randint(0, len(colors) - 1) for _ in range(N)]


def decode(chromosome):
    # Print the color name of each city
    for node in nodes:
        print(f"  City {node}: {colors[chromosome[gene_of[node]]]}")


# ---------- Fitness (lower is better) ----------
def count_bad_edges(chromosome):
    # Count the edges that join two cities of the same color
    bad_edges = 0
    for a, b in edges:
        if chromosome[gene_of[a]] == chromosome[gene_of[b]]:
            bad_edges += 1
    return bad_edges


def count_colors(chromosome):
    # A set keeps each color only once
    return len(set(chromosome))


def fitness(chromosome):
    # No bad edges is a must, few colors is a wish
    return count_bad_edges(chromosome) * PENALTY + count_colors(chromosome)


# ---------- Try it ----------
print_map()
print()
print_colors()

# Use the same random numbers on every run
random.seed(1)

# Make a small population of 6 random chromosomes
population = [random_chromosome() for _ in range(6)]

print("\nPopulation:")
for c in population:
    print(f"  {c}  bad edges = {count_bad_edges(c)}  colors = {count_colors(c)}"
          f"  fitness = {fitness(c)}")

# Lower is better, so the best chromosome has the smallest fitness
best = min(population, key=fitness)
print(f"\nThe best coloring in this population (fitness = {fitness(best)}):")
decode(best)
```

Output

```
The map (how the cities are connected):
  City 1 -> [2, 3, 4, 5, 7]
  City 2 -> [1, 3, 4, 5, 6, 7]
  City 3 -> [1, 2, 5, 6]
  City 4 -> [1, 2, 5, 6]
  City 5 -> [1, 2, 3, 4, 6, 7]
  City 6 -> [2, 3, 4, 5, 7]
  City 7 -> [1, 2, 5, 6]

The colors we can use (gene value = color):
  0 = Red
  1 = Green
  2 = Blue
  3 = Yellow
  4 = Purple
```

```
  5 = Orange
  6 = Pink

Population:
  [1, 4, 6, 6, 6, 0, 2]  bad edges = 2  colors = 5  fitness = 25
  [0, 3, 6, 3, 3, 5, 3]  bad edges = 5  colors = 4  fitness = 54
  [6, 1, 0, 3, 0, 6, 3]  bad edges = 1  colors = 4  fitness = 14
  [3, 4, 6, 6, 0, 5, 3]  bad edges = 1  colors = 5  fitness = 15
  [2, 5, 6, 1, 4, 0, 2]  bad edges = 1  colors = 6  fitness = 16
  [0, 0, 0, 5, 4, 0, 3]  bad edges = 5  colors = 4  fitness = 54

The best coloring in this population (fitness = 14):
  City 1: Pink
  City 2: Green
  City 3: Red
  City 4: Yellow
  City 5: Red
  City 6: Pink
  City 7: Yellow
```

Notes on the code:

- `gene_of[node]` gives the gene number of a city, for example `gene_of[1]` is 0. With it, the code works for any list of cities, even cities with names.
- `len(set(chromosome))` counts the different colors. A set keeps each value only once.
- `min(population, key=fitness)` gives back the chromosome with the smallest fitness. This is the best one, because lower is better.
- `neighbors()` reads the edge list and finds all cities connected to one city. `print_map()` uses it to show the whole map.
- `count_bad_edges()` and `count_colors()` each do one job. `fitness()` only puts the two numbers together.

Look at the population. The best chromosome has only 1 bad edge, but its fitness is 14. A correct coloring with 4 colors has fitness 4. Because of the penalty, a correct coloring always wins against a wrong one.

Try it yourself

Change `PENALTY` to 1. Can a wrong coloring now get the same fitness as a correct coloring with 4 colors? Hint: think of 3 colors and 1 bad edge. Why is this bad?

Solution 2: The Inheritance

1. Chromosome representation: binary

Use **one bit per item**, always in the same order of items:

- `0` means the item goes to Lina.
- `1` means the item goes to Omar.

So the chromosome has **12 genes**, one for each item. Here is an example:

Item	Car	Piano	Watch	Paint.	Carpet	Ring	Books	Sew.	Table	Bike	Cam.	Coins
Gene	0	0	1	1	1	1	0	1	1	0	1	1
Owner	L	L	O	O	O	O	L	O	O	L	O	O

Why is this a good representation?

- Each item has exactly **two** possible owners, and a bit has exactly **two** values. The match is perfect.
- Every chromosome is a real split: each item goes to exactly one person, and nothing is lost.

A third grandchild: does binary still work? No. One bit has only 2 values, but now every item has 3 possible owners. A first idea is to use 2 bits per item:

Bits	Meaning
00	Lina
01	Omar
10	Sara
11	??? Nobody. This person does not exist.

This creates invalid chromosomes, because any item coded `11` belongs to nobody. The better choice is **integer genes** with values 0, 1 or 2 (Lina, Omar, Sara). The chromosome still has 12 genes, and every chromosome is a real split again.

2. Fitness function

```
fitness   = | total of Lina - total of Omar |      (2 grandchildren)
fitness_3 = biggest total - smallest total         (3 grandchildren)
```

MIN Lower is better. 0 means a perfectly fair split.

Why this fitness? It is exactly the goal of the problem, written in money. A bigger gap means a less fair split. We use `abs()` because we only care about the size of the gap, not who has more. With three grandchildren we use the same idea: the gap between the richest and the poorest one.

3. Python code

2_inheritance.py

```
"""
Problem 2: The Inheritance

A grandfather leaves 12 items to his grandchildren.
Split the items so the totals are as close as possible.

Representation: binary, one gene per item (0 = Lina, 1 = Omar)
                for 3 grandchildren: integer (0 = Lina, 1 = Omar, 2 = Sara)
Fitness:        MINIMIZE  | total of Lina - total of Omar |   (0 = perfectly fair)

Run:  python 2_inheritance.py
"""

import random
```

```
# The 12 items and their values
items = {
    "Car": 12000, "Piano": 7000, "Watch": 1500, "Painting": 4300,
    "Carpet": 2100, "Gold ring": 3200, "Books": 900, "Sewing machine": 450,
    "Dining table": 1800, "Bicycle": 350, "Camera": 1250, "Coin collection": 5600,
}

# The items in a fixed order: gene i is always about item i
names = list(items.keys())

# One gene per item, so the chromosome has 12 genes
N = len(names)


# ---------- Representation ----------
def random_chromosome():
    # Give every item to Lina (0) or Omar (1) at random
    return [random.randint(0, 1) for _ in range(N)]


def decode(chromosome):
    # Print the items of each grandchild
    print("  Lina:", [n for n, g in zip(names, chromosome) if g == 0])
    print("  Omar:", [n for n, g in zip(names, chromosome) if g == 1])


# ---------- Fitness (lower is better) ----------
def totals(chromosome):
    # Add up the value of the items that each grandchild gets
    lina = sum(items[n] for n, g in zip(names, chromosome) if g == 0)
    omar = sum(items[n] for n, g in zip(names, chromosome) if g == 1)
    return lina, omar


def fitness(chromosome):
    # The gap between the two totals; abs() removes the minus sign
    lina, omar = totals(chromosome)
    return abs(lina - omar)


# ---------- Three grandchildren: Lina, Omar and Sara ----------
def random_chromosome_3():
    # Give every item to Lina (0), Omar (1) or Sara (2) at random
    return [random.randint(0, 2) for _ in range(N)]


def fitness_3(chromosome):
    # Add up the value of the items that each grandchild gets
    money = [0, 0, 0]
    for n, g in zip(names, chromosome):
        money[g] += items[n]
    # The gap between the richest and the poorest grandchild
    return max(money) - min(money)


# ---------- Try it ----------
# Use the same random numbers on every run
random.seed(4)

# Make a small population of 6 random chromosomes
population = [random_chromosome() for _ in range(6)]

print("Population:")
for c in population:
    lina, omar = totals(c)
    print(f"  {''.join(map(str, c))}  Lina = {lina:>5}  Omar = {omar:>5}"
          f"  gap = {fitness(c):>5}")

# Lower is better, so the best split has the smallest gap
best = min(population, key=fitness)
```

```
print(f"\nThe best split in this population (gap = {fitness(best)}):")
decode(best)

print("\nThree grandchildren (0 = Lina, 1 = Omar, 2 = Sara):")
for _ in range(2):
    c = random_chromosome_3()
    print(f"  {''.join(map(str, c))}  gap = {fitness_3(c)}")
```

Output

```
Population:
  010110000110  Lina = 25450  Omar = 15000  gap = 10450
  011001001100  Lina = 26600  Omar = 13850  gap = 12750
  111011000110  Lina = 13050  Omar = 27400  gap = 14350
  101101111100  Lina = 15950  Omar = 24500  gap =  8550
  110001111000  Lina = 15100  Omar = 25350  gap = 10250
  101101110100  Lina = 17750  Omar = 22700  gap =  4950

The best split in this population (gap = 4950):
  Lina: ['Piano', 'Carpet', 'Dining table', 'Camera', 'Coin collection']
  Omar: ['Car', 'Watch', 'Painting', 'Gold ring', 'Books', 'Sewing machine', 'Bicycle']

Three grandchildren (0 = Lina, 1 = Omar, 2 = Sara):
  201220010110  gap = 14950
  012012210111  gap = 12500
```

Notes on the code:

- `totals()` adds up the money of each grandchild. `fitness()` uses it and gives back the gap. Each function does one job.
- `''.join(map(str, c))` turns the list of genes into one short text, like `010110000110`, so it is easy to read.
- For three grandchildren, only `random_chromosome_3()` and `fitness_3()` change. The idea of the fitness stays the same: the smaller the gap, the better.

Look at the population. The gaps go from 4,950 to 14,350. Even the best split is still far from fair, but now we can see which split is closer to the goal.

Try it yourself

The total value of all 12 items is 40,450. Can the gap ever be 0? Hint: every value is a multiple of 50. What is the smallest gap that is possible?

8. Summary

Problem	Representation	One gene means	Fitness	MAX / MIN
Graph coloring	Integer (0–6)	The color of this node	bad edges × 10 + colors used	MIN
Inheritance	Binary (3 heirs: integer 0–2)	Who gets this item	| Lina − Omar |	MIN

Key message

1. Decide what one gene means. If you can write a clear `decode` function, your representation is clear.

2. Decide if your fitness is **MAX** or **MIN**. The goal of the problem tells you.

Good Luck